

Dr. Mahmoud Alnamoly

x

05

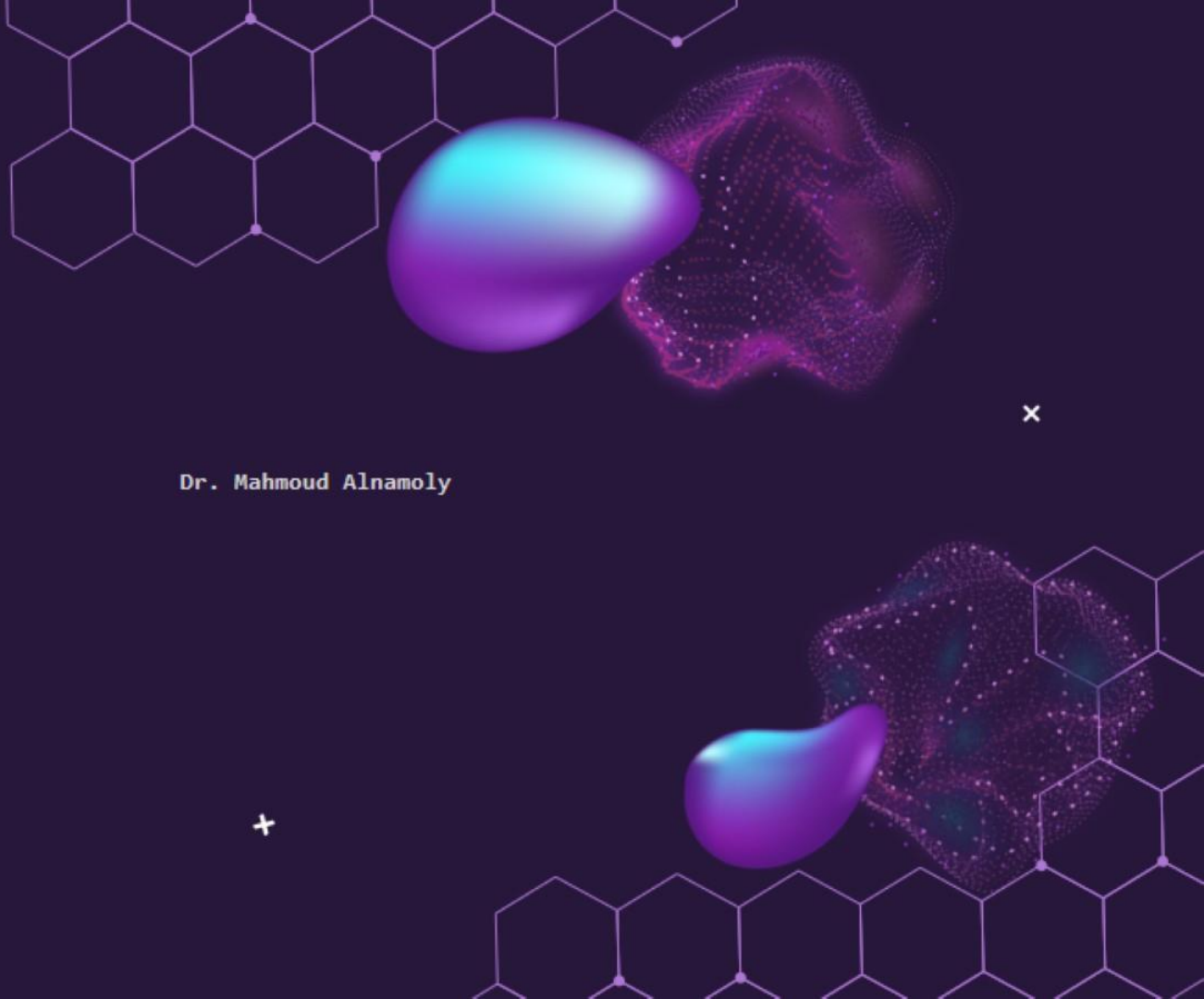
Stack

Dr. Mahmoud Alnamoly

x

+

Dr. Mahmoud Alnamoly

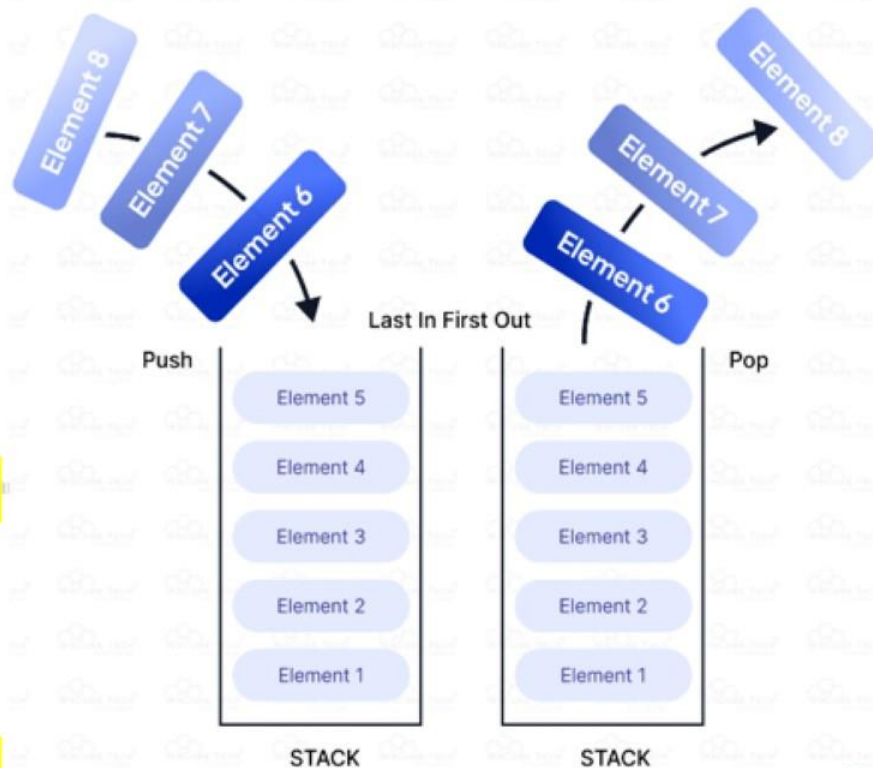


Stack ?

A stack in data structures is a **linear** collection of elements that follows a particular order in which the operations of addition and removal of elements are performed.

The order is referred to as "**Last In, First Out**" (**LIFO**). This means that the last element added to the stack is the first one to be removed.

Stacks are crucial for managing **function calls** in almost all programming environments, helping maintain the order of operations and execution seamlessly.



Understanding LIFO Principle in Stack

Dr. Mahmoud Alnamoly

The Last In, First Out (LIFO) principle dictates that the last element added to the stack is the first one to be removed.

In practice, a stack, which operates on the LIFO principle, supports mainly two operations:

Dr. Mahmoud Alnamoly

Push:

Adds an element to the top of the stack. This element becomes the most accessible item and will be the first to be removed.

Pop:

Removes the top element from the stack, which is the item that was most recently added.

Dr. Mahmoud Alnamoly

Stack operations

Dr. Mahmoud Alnamoly

We need to follow certain operations that are essential for managing data within a stack:

- ✓ Push
- ✓ Pop
- ✓ Peek (Top)
- ✓ IsEmpty
- ✓ IsFull
- ✓ Display
- ✓ Getsize
- ✓ Getcapacity
- ✓ resize

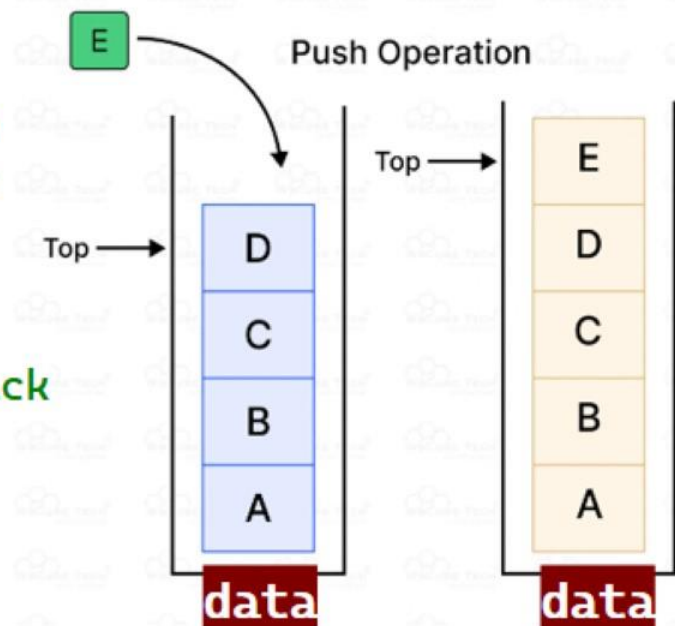
Dr. Mahmoud Alnamoly

✓ Push

The push operation adds an element to the top of the stack. This is the primary way to insert data into the stack.

// Function to add an element x to the stack

```
bool push(T val) {  
    if (IsFull()) {  
        std::cout << "Stack Overflow\n";  
        return false;  
    }  
    else {  
        data[++currenttop] = val;  
        std::cout << val << " pushed into stack\n";  
        return true;  
    }  
}
```



✓ Pop

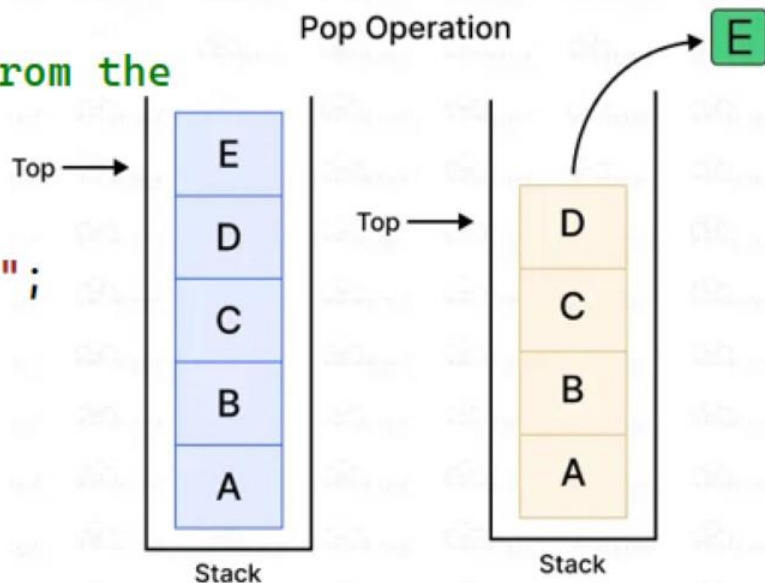
Dr. Mahmoud Alnamoly

The pop operation removes the top element from the stack and returns it. This operation follows the LIFO principle, ensuring that the last element added is the first to be removed.

```
// Function to remove the top element from the stack
```

```
int pop() {  
    if (IsEmpty()) {  
        std::cout << "Stack Underflow\n";  
        return 0;  
    }  
    else {  
        int val = data[currenttop--];  
        return val;  
    }  
}
```

Dr. Mahmoud Alnamoly

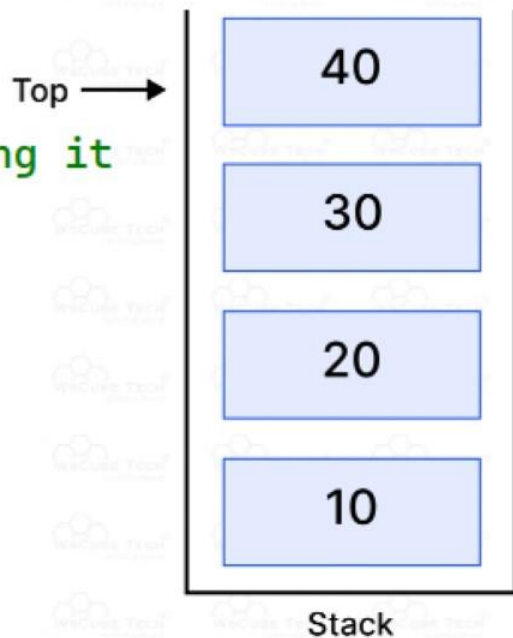


✓ Peek/Top

The peek or top operation retrieves the top element of the stack without removing it. This allows you to see what is at the top of the stack without modifying its content.

// Function to get the top element without removing it

```
int peek() {  
    if (isEmpty()) {  
        std::cout << "Stack is Empty\n";  
        return 0;  
    }  
    else {  
        int val = data[currenttop];  
        return val;  
    }  
}
```



Top Element = 40

Stack Implementation using dynamic array

```
#include <iostream>
template<class T>
class Stack {
private:
    // Dynamic array to store stack elements
    T* arr;
    // Maximum capacity of the stack
    int capacity;
    // Index of the currenttop element
    int currenttop;
```

Stack

```
arr: *T
capacity: Size_t
currtop: Int
```

```
push(T element) : void
pop() : void
Peek() : T
isEmpty() : bool
Size(): int
Capacity(): int
print(): void
```

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

```
public:
    // Constructor to initialize stack
    Stack(int size = 2) {
        arr = new T[size];
        capacity = size;
        currenttop = -1;
    }

    // Destructor to free the allocated memory
    ~Stack() {
        delete[] arr;
    }
```

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

```
// Helper function to resize the array
void resize(int newCapacity) {
    T* newArr = new T[newCapacity];
    for (int i = 0; i <= currenttop; ++i) {
        newArr[i] = arr[i];
    }
    delete[] arr;
    arr = newArr;
    capacity = newCapacity;
}
```

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

```
// Function to add an element to the stack
void push(T val) {
    if (currenttop + 1 == capacity) {
        // Double the capacity if stack is full
        resize(2 * capacity); Alnamoly
    }
    arr[++currenttop] = val;
    std::cout << val << " pushed into stack.\n";
}
```

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

```
// Function to remove the currenttop element from the stack
void pop() {
    if (currenttop == -1) {
        std::cout << "Stack Underflow.\n";
        return;
    }
    std::cout << arr[currenttop] << " popped from stack.\n";
    currenttop--;

    // Shrink the array size if it's only 1/4 full
    if (currenttop + 1 <= capacity / 4 && capacity > 2) {
        resize(capacity / 2);
    }
}
```

Dr. Mahmoud Alnamoly

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

```
// Function to get the currenttop element
```

```
T peek() {  
    if (currenttop == -1) {  
        std::cout << "Stack is Empty.\n";  
        return -1;  
    }  
    return arr[currenttop];  
}
```

id Alnamoly

```
// Function to check if the stack is empty
```

```
bool isEmpty() {  
    return currenttop == -1;  
}
```

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

```
// Function to get the size of the stack
```

```
int size() {  
    return currenttop + 1;  
}
```

```
// Function to get the current capacity of the stack
```

```
int getCapacity() {  
    return capacity;  
}
```

Dr. Mahmoud Alnamoly

```
void print() {  
    std::cout << "---- Stack elements ----\n";  
    for (int i = 0; i <= currenttop; ++i) {  
        std::cout << arr[i] << " ";  
    }  
    std::cout << "\n";  
}
```

Dr. Mahmoud Alnamoly

Stack Implementation using dynamic array

```
int main() {  
    Stack<int> s;  
    Stack<double> ds;  
    Stack<std::string> ss;  
  
    // Push elements into the stack  
    s.push(10);  
    s.push(20);  
    s.push(30);  
    s.push(40);  
  
    std::cout << "currenttop element is:" << s.peek() << std::endl;  
  
    // Pop elements from the stack  
    s.pop();  
    s.pop();  
}
```

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

```
std::cout << "currenttop element after popping two elements: "  
<< s.peek() << std::endl;
```

```
// Push more elements  
s.push(50);
```

Dr. Mahmoud Alnamoly

```
std::cout << "Size of stack: " << s.size() << std::endl;  
std::cout << "Capacity of stack: " << s.getCapacity() <<  
std::endl;  
  
s.print();  
return 0;  
}
```

Dr. Mahmoud Alnamoly

Stack Implementation using dynamic array

Dr. Mahmoud Alnamoly

Microsoft Visual Studio Debug Console

```
10 pushed into stack.  
20 pushed into stack.  
30 pushed into stack.  
40 pushed into stack.  
currenttop element is: 40  
40 popped from stack.  
30 popped from stack.  
currenttop element after popping two elements: 20  
50 pushed into stack.  
Size of stack: 3  
Capacity of stack: 4  
---- Stack elements ----  
10 20 50
```

Queue

>>> Next Lecture